

VERİ ORGANİZASYONU

B ve B+ Ağaçlarının Günümüzde Kullanım Alanları

- ADI SOYADI: YILDIZ ŞAHİN
- NUMARA: 02240224176

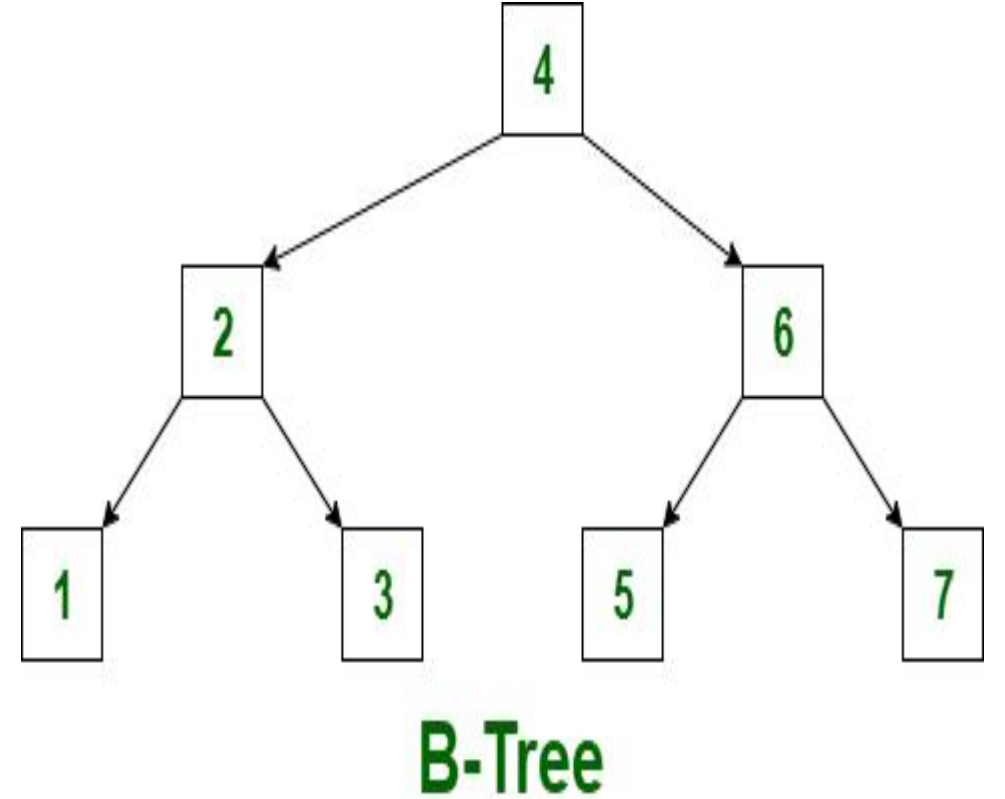
Veri Tabanlarında Kullanımı

B Ağaçları:

Veritabanı İndeksleme (Indexing): Tablo taraması yapmadan kayıtlara doğrudan ulaşmak için oluşturulan birincil indekslerin (Primary Index) temelini oluşturur.

İkincil Depolama Optimizasyonu: Sabit diskler (HDD) veya SSD'ler üzerindeki büyük veri bloklarının okunmasını en aza indirerek disk I/O performansını artırır.

Aralık Sorguları (Range Queries): Sıralı yapısı sayesinde "A ile Z arasındaki" veya belirli bir tarih aralığındaki verilerin verimli bir şekilde çekilmesine olanak tanır.



Veri Tabanlarında Kullanımı

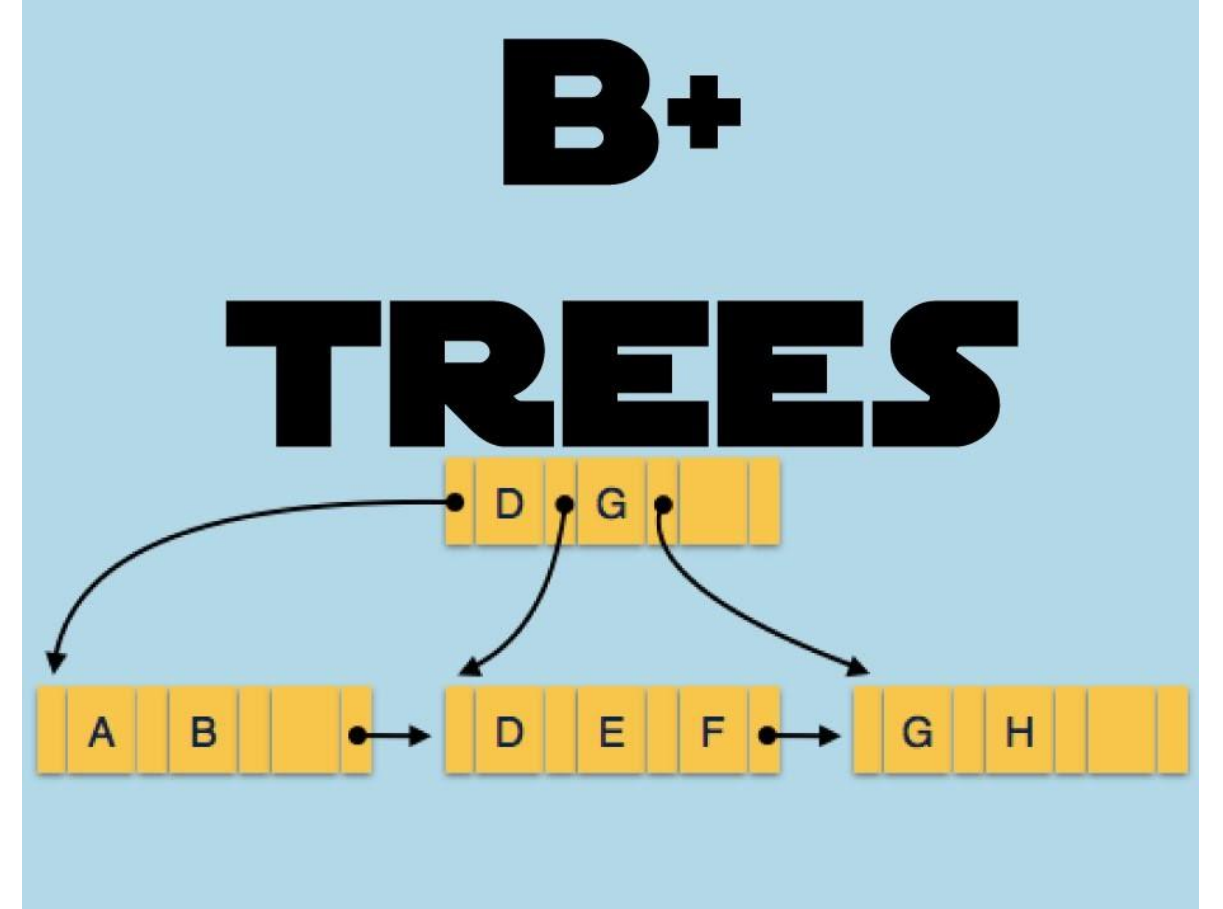
B+ Ağaçları:

1. Veri Tabanı İndeksleme (Database Indexing):

Temel İndeksler: Birincil anahtar (Primary Key) sütunlarında, verilerin diske sıralı ve hızlı bir şekilde kaydedilmesini / aranmasını sağlar. İkincil İndeksler (Secondary Indexes): Tablodaki sıralı olmayan (örneğin e-posta veya soyisim) sütunlar üzerinden hızlı sorgulama yapılmasına imkân tanır.

2. Aralık Sorguları (Range Queries):

B+ ağaçlarında sadece yaprak (leaf) düğümler gerçek veriye veya veri adreslerine işaret eder ve bu yapraklar birbirine yatay bağlantılarla (linked list) bağlıdır. Bu sayede "Yaşı 20 ile 30 arasında olanlar" veya "01.01.2026 ile 31.01.2026 arasındaki işlemler" gibi aralık sorguları inanılmaz derecede hızlı çalışır.



Veri Tabanlarında Kullanımı

B+ Ağaçları:

3. Disk I/O Optimizasyonu:

Veri tabanları veriyi sabit disklerden sayfa (page/block) blokları halinde okur. B+ ağaçları çok yüksek bir dallanma katsayısına (fanout) sahiptir; bu da ağacın boyunu (yüksekliğini) kısa tutar.

Bu sayede aranan bir veriye ulaşmak için gereken disk okuma (I/O) işlemi minimize edilir.

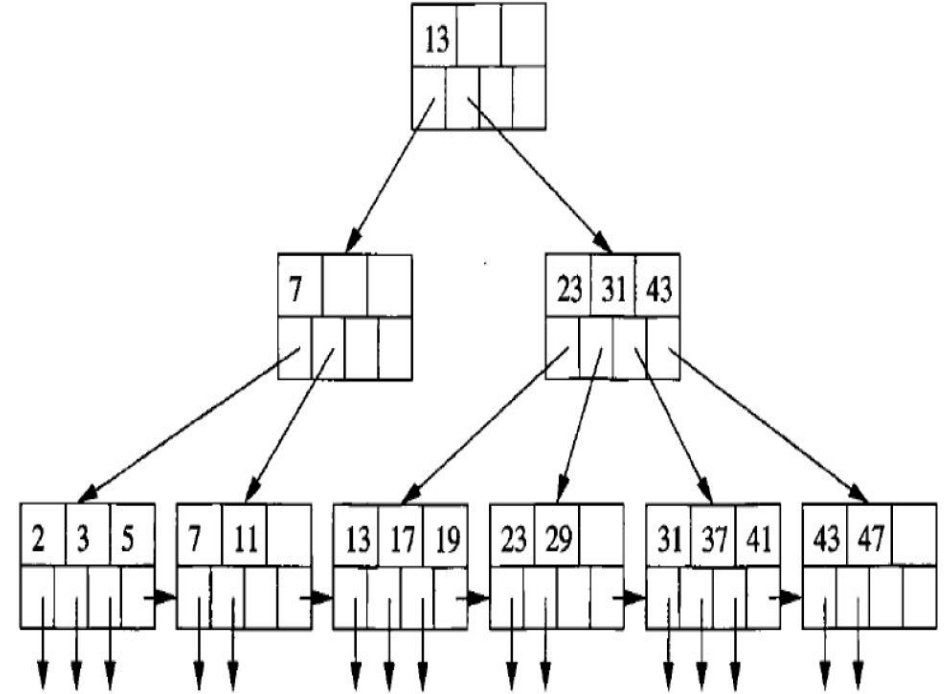
4. İşlem (Transaction) ve Günlük (Log) Yönetimi:

Finansal veri tabanları ve bankacılık sistemleri, ardışık ekleme/silme operasyonlarını ve işlem günlüklerini (transaction logs) yönetmek için B+ ağaçlarından faydalanır.

5. Veri Ambarı (Data Warehousing):

Çok büyük veri kümelerinin bulunduğu veri ambarlarında, verimli toplama (aggregation), filtreleme ve sıralama işlemleri için B+ ağacı indeksleri kullanılır.

EXAMPLE: B+-TREE



Dosya Sistemlerinde Kullanımı

B Ağaçları:

- **Disk Odaklı Veri Erişimi:** Sabit disklerde (HDD) ve SSD'lerde veri okuma/yazma bloklar (sektörler) halinde gerçekleşir. B-ağaçları yüksek dallanma faktörüne sahip oldukları için veriyi disk bloklarına tam oturacak şekilde düzenlerler. Bu, disk erişim sayısını (I/O) en aza indirir.
- **Hızlı Arama, Ekleme ve Silme:** Dosya sayısı milyonları bulsa bile, B-ağaçları sayesinde belirli bir dosyayı veya dizini bulmak $\mathcal{O}(\log n)$ karmaşıklığında, son derece hızlı bir şekilde gerçekleşir.
- **Dizin Yönetimi (Indexing):** Bir klasörün içerisindeki binlerce dosyayı doğrusal olarak aramak yerine B-ağacı yapısı kullanılarak klasör içeriği hiyerarşik ve sıralı indekslenir.
- **B-Ağaçları Kullanan Popüler Dosya Sistemleri:**
 - **Btrfs (B-tree File System):** Adını doğrudan B-ağaçlarından alır. Veri ve meta verileri tamamen B-ağaçları ile yönetir. Dosya sistemi düzeyinde anlık görüntü (snapshot), sıkıştırma ve büyük ölçeklenebilirlik sağlar.
 - **NTFS (New Technology File System):** Microsoft'un geliştirdiği NTFS, dosya ve klasör kayıtlarını organize etmek ve bunlara hızlı erişim sağlamak için B+ ağacı (B-tree türevi) yapısını kullanır.
 - **HFS+:** Apple'ın macOS işletim sistemlerinde kullandığı dosya sistemidir; dosyaların ve dizinlerin izini sürmek için B-ağaçlarından faydalanır.
 - **ReiserFS:** Linux dünyasında meta verileri doğrudan B-ağaçları ile indeksleyen ve özellikle küçük boyutlu birçok dosyanın saklandığı durumlarda yüksek performans veren öncü bir dosya sistemidir.

Dosya Sistemlerinde Kullanımı

B+ Ağaçları:

- Çalışma Mantığı
- **Yaprak Düğümler (Leaf Nodes):** Gerçek dosya veya dizin verileri, yalnızca sıralı bağlı liste şeklinde tasarlanmış yaprak düğümlerde tutulur.
- **İç Düğümler (Internal Nodes):** Üst seviyelerdeki iç düğümler (dallanma noktaları) ise sadece yönlendirme (indeks) anahtarlarını barındırır. Bu sayede tek bir okuma işleminde çok daha fazla anahtar taranabilir.
- **Bağlantılı Liste Yapısı:** Yaprak düğümlerin birbirine çift yönlü bağlı olması, dosya sisteminde sıralı işlemleri (ardışık okuma/yazma) inanılmaz derecede hızlandırır.

Dosya Sistemlerinde Kullanımı

B+ Ağaçları:

- **Disk Bloklarıyla Uyum:** Dosya sistemleri veriyi sektör veya bloklar halinde (örneğin 4 KB) okur. B+ ağaçları bu disk blokları üzerinde gezinmek için optimize edilmiştir.
- **Sabit Arama Süresi:** Dizin ağacı büyüse bile, arama, ekleme ve silme işlemleri zaman karmaşıklığında ve öngörülebilir hızda gerçekleşir.
- **Daha Az Disk Okuma/Yazma (I/O):** Yönlendirme bilgisi iç düğümlerde, verinin kendisi yapraklarda olduğundan, ağaç daha sığ kalır. Bu da aranan veriye ulaşmak için gereken disk erişim sayısını en aza indirir.
- **Kullanan Dosya Sistemleri:** APFS, NTFS, ReiserFS, XFS

B ve B+ Ağaçlarının Arama Motorları ve Büyük Veri Sistemlerinde Kullanımı:

- **Neden Büyük Veritabanları ve Arama Motorları İçin Vazgeçilmezdir?**
- **Blok Tabanlı Okuma:** Sabit diskler (HDD) ve SSD'ler verileri küçük parçalar halinde değil, bloklar (page/cluster) halinde okur. B-ağaçları, düğümleri (node) disk bloklarıyla aynı boyutta olacak şekilde geniş tasarlanır. Bu, tek bir disk okuma işlemiyle çok daha fazla veriye ve alt dala erişilmesini sağlar.
- **Dengeli (Balanced) Yapı:** Ağacın tüm yaprak düğümleri aynı seviyededir. Bu sayede veriler derinlikte kaybolmaz; en kötü senaryoda bile arama süresi tahmin edilebilirdir.
- **Disk I/O Azaltımı:** Düğümler ikiden fazla (geniş) çocuk tutabildiği için ağacın boyu (derinliği) kısa kalır. Bu da fiziksel diske gitme gereksinimini (disk I/O) büyük ölçüde düşürür.

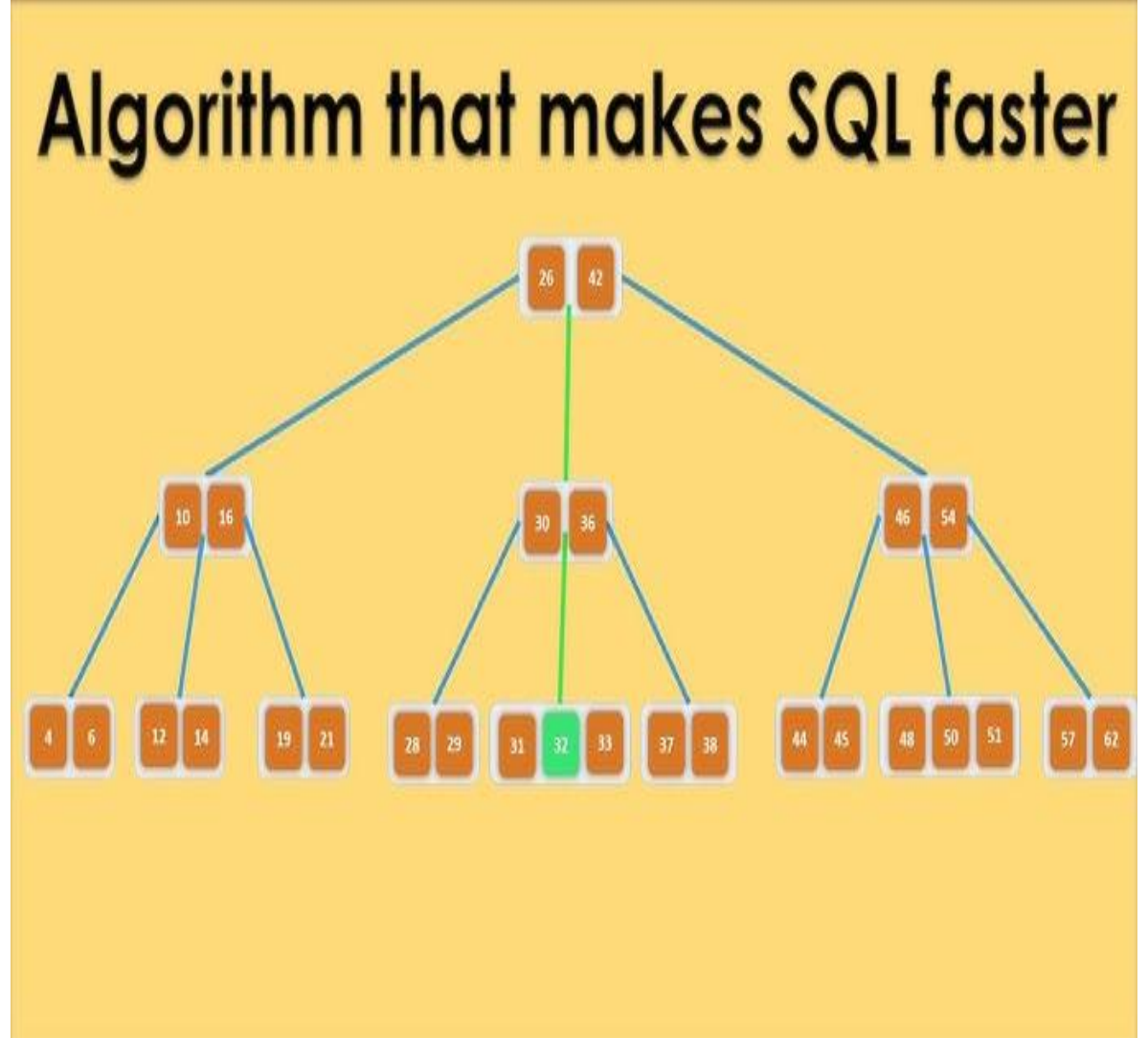
!! MySQL ve PostgreSQL gibi sistemlerde kayıtları anahtarlara göre bulmak için B-tree (ve türevi B+ tree) indeksleri varsayılan olarak kullanılır.

B ve B+ Ağaçları Neden Modern Sistemlerde Tercih Edilmektedir?

Geleneksel ikili arama ağaçları (BST, AVL, Red-Black) teoride çok iyi olsalar da, işin içine fiziksel disk geometrisi ve bellek hiyerarşisi girdiğinde yetersiz kalırlar.

Geniş ve Basık (Dengeli) Yapı: İkili ağaçlar her düğümde sadece bir anahtar tutar ve çok derindir (yüksektir). B ve B+ ağaçları ise her düğümde yüzlerce, hatta binlerce anahtar tutabilir. Bu sayede ağacın yüksekliği genellikle 3 veya 4 katmanla sınırlı kalır. milyonlarca veri arasında arama yaparken sadece 3-4 disk okuması yeterli olur.

Sayfa Boyutu ile Uyum: İşletim sistemleri diskten veriyi bayt bayt değil, "Sayfa" (genellikle 4KB veya 8KB) adı verilen bloklar halinde okur. B/B+ ağaçlarının bir düğümünün boyutu, tam olarak bu disk sayfa boyutuna denk gelecek şekilde ayarlanır. Böylece tek bir disk I/O işleminde yüzlerce dallanma alternatifi belleğe yüklenmiş olur.



Hastane Arşivi Örneği

B Ağacı:

- B-Tree sisteminde arşiv binası çok katlıdır. Bu binanın her katında hem "Hangi dosya için hangi kata gitmeniz gerektiğini söyleyen danışma masaları (indeks)" hem de "Bizzat hastaların tedavi dosyaları (veri)" yan yana bulunur.
- Doktor, Ayşe Yılmaz isimli hastanın dosyasını istiyor. Arşiv görevlisi binaya girer:
- **1. Kat (Kök Düğüm):** Katın girişindeki panoda der ki: "A-Ç arası hastalar bu katta, D-M arası için 2. kata, N-Z arası için 3. kata çıkın."
- Görevli panoya bakar. Eğer aranan hasta Ahmet Yılmaz ise, şanslı gündür! Ahmet'in dosyası tam o panonun arkasındaki raftadır. Dosyayı alır ve üst katlara hiç çıkmadan işini bitirir.
- Eğer aranan hasta Kemal Öztürk ise, görevli 1. kattaki yönlendirmeye bakarak "D-M arası" yönergesini takip eder ve 2. kata çıkar. 2. kata geldiğinde yine bazı dosyaları ve daha detaylı yönlendirmeleri bir arada bulur.
- **B-Tree Özeti:** Aradığınız hasta dosyası üst katlarda veya ara katlarda karşınıza çıkabilir. Bulduğunuz anda işlem biter.

Hastane Arşiv Örneği

B+ ağacı:

- Modern veri tabanlarının kullandığı B+ Tree sisteminde ise kurallar nettir: Üst katlarda asla tek bir hastanın bile dosyası (verisi) bulunmaz. Üst katlar sadece ve sadece devasa birer yönlendirme tabelasıdır. Bütün hastaların dosyaları istisnasız en alt katta (zemin katta) yan yana dizilmiştir.
- Doktor yine bir hasta dosyası istiyor. Görevli arşive girer:
- Üst Katlar (Sadece İndeks): Girişte ve üst katlarda sadece rehber memurlar veya tabelalar vardır. Tabelada "A-K arası hastalar için sol koridora, L-Z arası için sağ koridora gidin" yazar. Bu katlardaki raflar tamamen boştur, sadece yön tabelaları barındırırlar. Hiçbir hastanın dosyası burada saklanmaz.
- En Alt Kat (Tüm Dosyalar Burada): Görevli tabelaları takip ederek en alt kata, yani ana depo salonuna iner. Milyonlarca hastanın dosyası alfabetik sırayla yan yana bu salondadır.

Hastane Arşivcisi Gözünden Özet Farklar

ÖZELLİK	B-Tree Arşivi	B+ Tree Arşivi
Dosyalar (Veri) Nerede?	Her katta (girişte, ortada veya en altta) bulunabilir.	Sadece ve sadece en alt katta bulunur. Üst katlar boştur, sadece tabeladır.
Tek Bir Hasta Aramak	Şanslıysanız ilk katta bulursunuz, arama çok hızlı biter.	Dosya en başta bile olsa, doğrulamak için mutlaka en alt kata inmeniz gerekir. (Süre hep sabittir).
Belli Bir Aralıktaki Hastaları Toplamak	Katlar arasında sürekli yukarı-aşağı mekik dokursunuz (Yavaş ve yorucu).	En alt kata bir kez inip, raflar boyunca yan yana yürüyerek tüm dosyaları hızla toplarsınız (Çok hızlı ve verimli).

Bu veri yapıları neden önemlidir?

- B ve B+ ağaçlarının bu kadar önemli olmasının sebebi, büyük miktardaki veriyi en az çabayla yönetebilmeleridir. Aradığınız veriyi daha az zaman harcayarak bulunmanızı sağlar. Bilgisayarı yormaz. Düzenlidir.

Büyük veri çağında neden ihtiyaç duyulmaktadır?

- Verilere çok hızlı erişim sağlar. Veriler arasında kaybolmamanızı sağlar. Veri eklenip silinirken dengeli yapının bozulmamasını sağlar.
- Veritabanı sistemlerinde (MySQL, Oracle gibi) indeksleme için çok uygundur.

Sizce gelecekte de kullanılmaya devam edecek midir?

Kullanım açısından çok fazla avantajı olduğu için gelecekte de kullanılacağını düşünüyorum. Ancak gelecekte yapay zekanın yardımıyla işlemlerin daha hızlı ve daha kolay şekilde gerçekleştirileceğini düşünüyorum.